

Enhancing Software Quality with Test Automation

- Facilitates cross-browser testing, ensuring compatibility across different browsers.
- Enables instant execution of test cases, Beneficial for early regression.
- less repetitive regression testing, enhancing software quality.
- Significantly reduces testing time compared to manual testing.
- Identifies defects early in the development cycle, preventing costly bug fixes in production.

Benefits of Test Automation

Cross-Browser Testing

Ensures compatibility across different browsers

Instant Execution

Allows for quick execution of test cases

Reduced Repetition

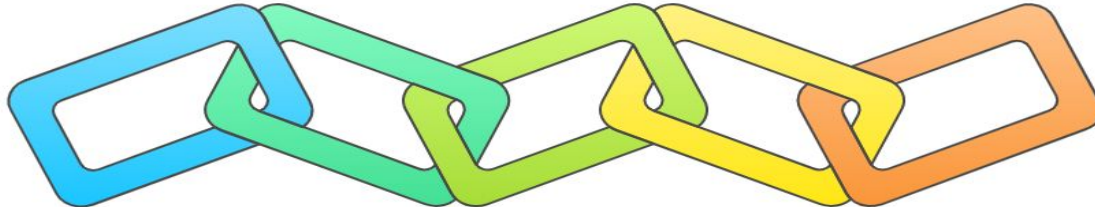
Minimizes repetitive regression testing

Time Efficiency

Significantly reduces testing time

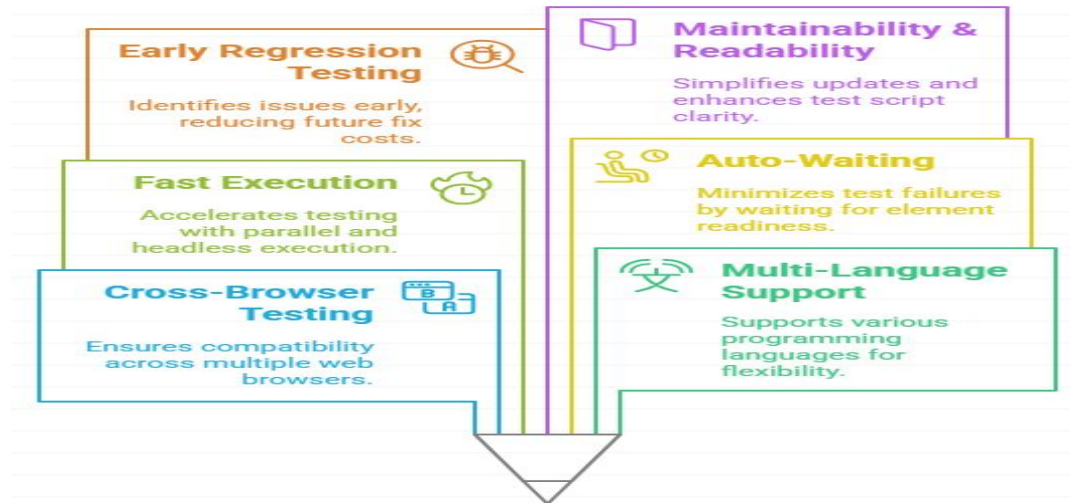
Early Defect Identification

Detects defects early in development



Boosting Test Efficiency with Playwright

- ✓ **Cross-browser testing:** Run tests in Chromium, Firefox, and WebKit.
- ✓ **Multi-language support:** Works with JavaScript, TypeScript, Python, C#, and Java.
- ✓ **Fast execution:** Parallel testing and headless mode improve performance.
- ✓ **Auto-waiting:** Reduces flakiness by waiting for elements to be ready.
- ✓ **Early regression testing:** Detects defects early in the development cycle, reducing costly fixes later
- ✓ **Maintainability:** Changes in the UI require updates only in the page object classes.
- ✓ **Readability:** Test scripts are easier to read and understand since they focus on the "what" rather than the "how."



Auto-generating test reports

Playwright provides built-in reporting tools like HTML and JSON reports for better test insights.

Q

All 10Passed 9✖ Failed 1Flaky 0Skipped 0

Project: chromium

4/1/2025, 8:17:03 PM Total time: 1.5m

▼ commonTest.test.ts

✖ Verify user can navigate to Brand Information and verify its correct display33.9s

commonTest.test.ts:53

✓ Verify that user is able to login with valid credentials22.4s

commonTest.test.ts:13

✓ Verify user can navigate to the "Brand" screen and verify screen displayed correctly25.7s

commonTest.test.ts:23

✓ Verify that the user can apply a filter and check the presence of records in the data grid.26.5s

commonTest.test.ts:32

✓ Verify user can Sort Brand records and verify sort order of records.25.9s

commonTest.test.ts:41

✓ Verify user can Add Brand with "valid data" and verify Success message30.4s

commonTest.test.ts:63

✓ Verify user cannot Add Brand with "invalid data" and verify error message33.3s

commonTest.test.ts:74

✓ Verify that the user can edit and save the details of an existing brand30.2s

commonTest.test.ts:85

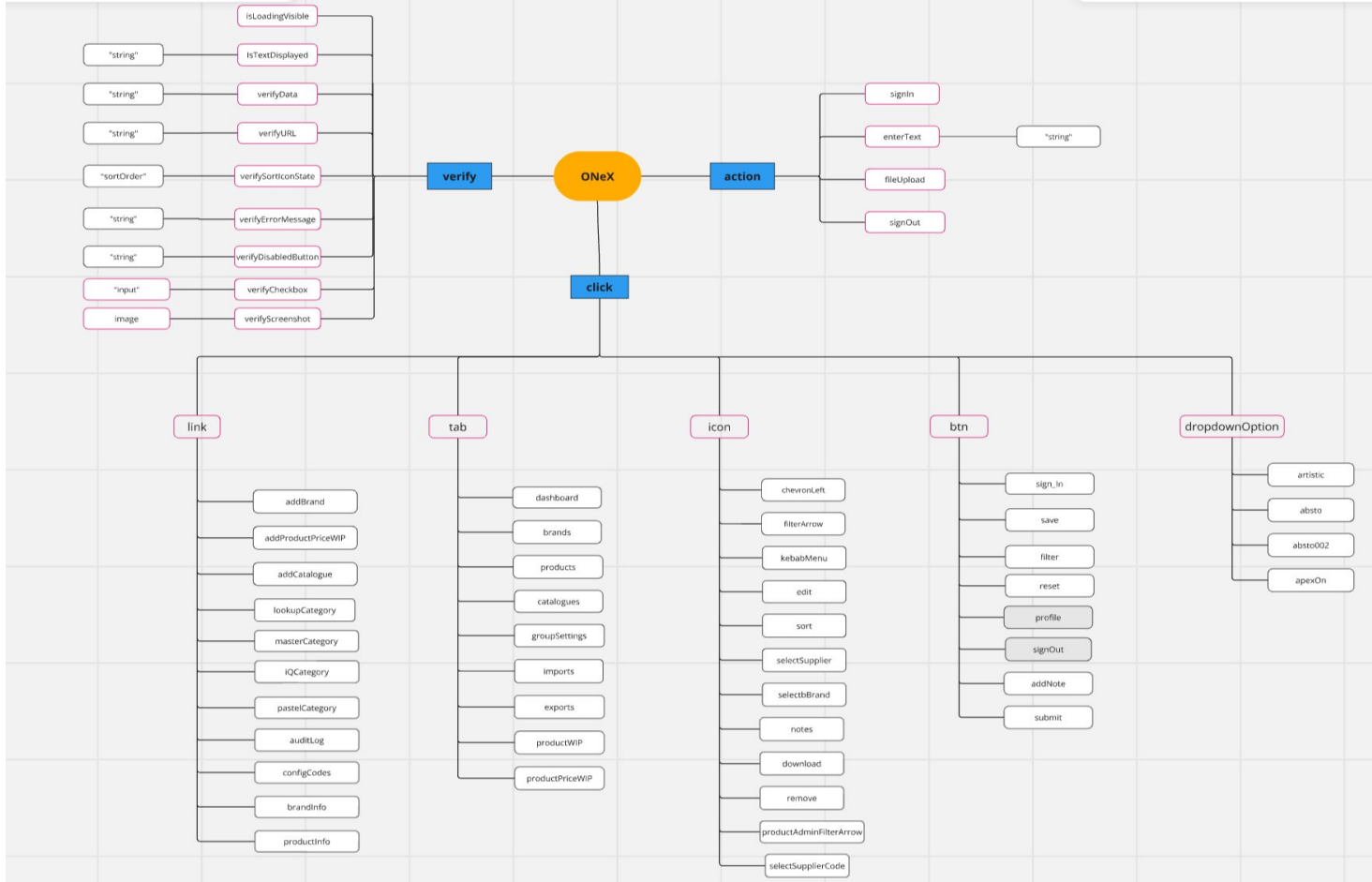
✓ Verify user can upload brand logo and verify success message26.1s

commonTest.test.ts:97

✓ Verify user can download uploaded brand logo file29.3s

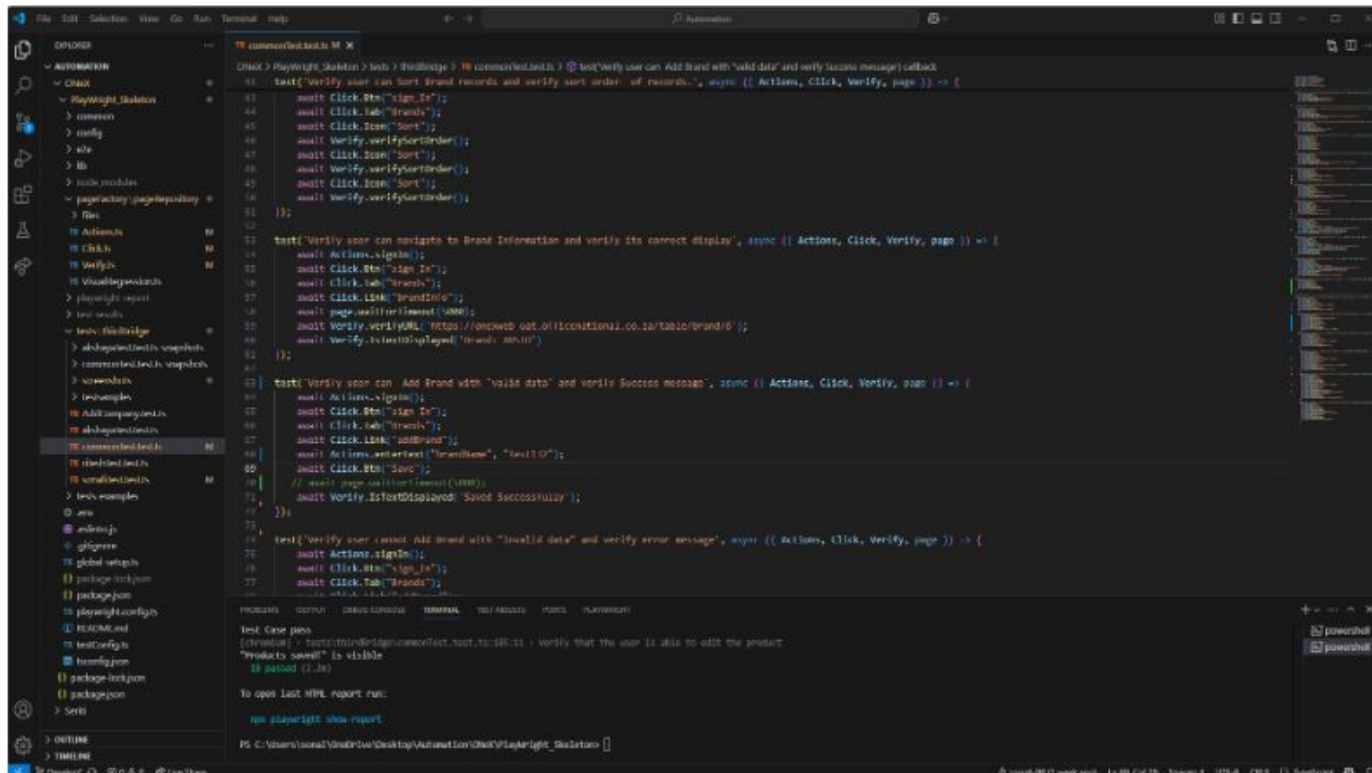
commonTest.test.ts:109

Benefits of This Approach (Based on Mind Map Approach)



Framework Structure

Our Playwright framework structure utilizes the Page Object Model (POM) for efficient and maintainable test automation. It describes the organization of page objects, classes, and test scripts, highlighting the benefits of this modular approach.



Page Objects

Each web page or component is represented by a separate class file. These classes contain methods that define user interactions, such as clicks, actions, and verifications.

Classes

We have dedicated classes for different functionalities:

- **Click** : Handles click actions on UI elements.
- **Actions** : Manages user interactions like form submissions, data entry, etc.
- **Verify** : Validates UI elements, content, and test outcomes.

This modular approach allows us to reuse common actions across different tests, enhancing efficiency and maintainability.

Test Scripts

Test cases are stored in different files, making them easy to manage. Each test script brings in the necessary page objects and action classes to run specific test scenarios. This setup keeps the test logic simple and focused on checking if the application works correctly, without getting mixed up with technical details.

```
// Login -----  
  
test('Verify that user is able to login with valid credentials', async ({ page, Actions, Click, Verify }) => {  
  await Actions.signIn();  
  await Click.Btn("sign_in");  
  await Verify.IsTextDisplayed("My OfficeNational");  
});
```